



Graph Data Structure

The Complete Deep-Dive Guide

□ 3 Graph Types

□ 2 Memory
Representations

□ Full Python Code

Table of Contents

Table of Contents	1
1. What Is a Graph?	2
1.1 Formal Definition	2
1.2 Real-World Examples	2
2. Types of Graphs	3
2.1 Undirected Graph	3
2.2 Directed Graph (Digraph)	3
2.3 Weighted Graph	3
3. Memory Representation of Graphs	4
3.1 Adjacency Matrix	4
3.2 Adjacency List	5
4. Adjacency Matrix Implementation (Python)	6
4.1 Class Structure	6
4.2 add_edge — Adding an Edge	7
4.3 remove_edge — Removing an Edge	7
4.4 print_graph — Visualizing the Matrix	7
4.5 Complete Code + Step-by-Step Trace	8
5. Adjacency List Implementation (Python)	9
5.1 Class Structure + add_vertex	9
5.2 add_edge — Both Directions	9
5.3 remove_edge	9
5.4 remove_vertex	10
5.5 print_graph	10
5.6 Complete Code + Testing	10
6. Complete Complexity Summary	11
7. Key Terminology Glossary	12

1. What Is a Graph?

From sets to connections — the math behind the data structure

1. What Is a Graph?

A Graph is a non-linear data structure made of two things: a set of Vertices (also called Nodes) and a set of Edges (connections between those vertices). Unlike Trees, graphs have no root and edges can go in any direction — or both directions at once.

1.1 Formal Definition

Mathematically, a Graph G is defined as:

```
G = (V, E)

V = Set of Vertices (Nodes)
E = Set of Edges (Connections between vertices)

Example:
V = {1, 2, 3, 4, 5}      ← 5 vertices
E = {(1,2), (2,4), (3,6)} ← 3 edges (ordered pairs)

Reading (1,2): vertex 1 is connected to vertex 2.
This forms: 1 --> 2 --> 4 and 3 --> 6
```

Key Insight: Trees vs Graphs

A Tree IS a special type of Graph — one that is connected, undirected, and has no cycles. Graphs are more general: they can have cycles, disconnected components, and bidirectional edges. Everything you know about Trees applies to Graphs, but Graphs have more possibilities!

1.2 Real-World Examples

Real-World System	Vertices	Edges	Graph Type
Google Maps / GPS	Intersections / Cities	Roads between them	Weighted + Directed
Facebook Friends	People (Users)	Friendships	Undirected
Twitter / Instagram	Users	Follows	Directed
Internet / Web	Web Pages	Hyperlinks	Directed
Airline Routes	Airports	Flight paths	Weighted + Directed
Computer Networks	Computers / Routers	Network cables	Undirected / Weighted

2. Types of Graphs

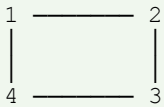
3 fundamental types every developer must know

2. Types of Graphs

2.1 Undirected Graph

In an Undirected Graph, edges have NO direction. If there is an edge between A and B, you can travel from A to B AND from B to A — it is a two-way road.

Undirected Graph — edges work BOTH ways:



Edge (1,2) means: 1 <---> 2 (can go either direction)

Edge (2,3) means: 2 <---> 3

Edge (1,4) means: 1 <---> 4

Edge (3,4) means: 3 <---> 4

Key: Adjacency matrix is SYMMETRIC for undirected graphs.

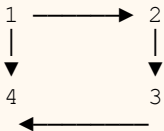
If $\text{mat}[i][j] = 1$, then $\text{mat}[j][i] = 1$ too.

Real-world example: Facebook friendship graph — if Alice is friends with Bob, Bob is also friends with Alice.

2.2 Directed Graph (Digraph)

In a Directed Graph, every edge has a direction — like a one-way street. An edge ($A \rightarrow B$) means you can travel from A to B, but NOT necessarily from B to A.

Directed Graph — edges are one-way arrows:



Edge (1,2): can go 1 --> 2, but NOT 2 --> 1

Edge (1,4): can go 1 --> 4, but NOT 4 --> 1

Edge (3,4): can go 3 --> 4, but NOT 4 --> 3

To add the reverse path, you must explicitly add a new edge.

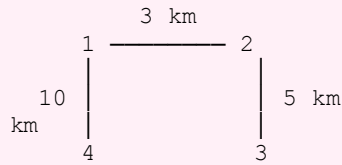
Key: Adjacency matrix is NOT symmetric for directed graphs.

Real-world example: Twitter follows — if you follow Elon Musk, he does NOT automatically follow you back.

2.3 Weighted Graph

In a Weighted Graph, each edge carries a numeric value — a weight. This weight typically represents cost, distance, time, or capacity. Weighted graphs can be directed OR undirected.

Weighted Graph — edges carry values:



Edge (1,2) = 3 ← distance 3 km
 Edge (1,4) = 10 ← distance 10 km
 Edge (2,3) = 5 ← distance 5 km

In matrix: `mat[src][dest] = weight` (not just 0 or 1)
 Use case: GPS finds SHORTEST path using these weights.

□ When to Use Which?

Undirected: social networks, road maps (2-way streets), peer-to-peer networks.

Directed: web page links, Twitter follows, task scheduling (dependency graphs).

Weighted: GPS/mapping, network routing (minimize latency), airline route costs.

Weighted + Directed: One-way streets with speed limits, airline pricing.

□ 3. Memory Representation of Graphs

Matrix vs List — tradeoffs every developer must know

3. Memory Representation of Graphs

There are two standard ways to store a graph in memory. Each has very different tradeoffs for memory and speed.

3.1 Adjacency Matrix

An Adjacency Matrix is a 2D array (grid) of size $V \times V$, where V is the number of vertices. `mat[i][j] = 1` means there is an edge from vertex i to vertex j . `mat[i][j] = 0` means no edge exists.

For a graph with 4 vertices, we create a 4×4 matrix:

	1	2	3	4
1	0	1	0	1
2	1	0	1	0
3	0	1	0	0
4	1	0	0	0

← Undirected (symmetric)

Reading: `mat[src][dest] = value`

`mat[1][2] = 1` → edge exists from vertex 1 to vertex 2

`mat[2][1] = 1` → edge exists from vertex 2 to vertex 1 (undirected = symmetric)

For Directed Graph:

`mat[1][2] = 1` → edge from 1 → 2 ONLY

`mat[2][1] = 0` → NO edge from 2 → 1

For Weighted Graph:

```
mat[1][2] = 3 → edge from 1 → 2 with weight 3
mat[1][4] = 10 → edge from 1 → 4 with weight 10
```

□ Dense vs Sparse Graphs

DENSE graph (many edges, many 1s in matrix) → USE Adjacency Matrix

SPARSE graph (few edges, mostly 0s in matrix) → DON'T USE Adjacency Matrix (memory waste!)

Example: 1000-vertex graph = $1000 \times 1000 = 1,000,000$ cells stored.

If only 500 edges exist, 999,500 cells are just wasted zeros!

Aspect	Adjacency Matrix
Space Complexity	$O(V^2)$ — always allocates $V \times V$ space
Add Edge	$O(1)$ — just set <code>mat[src][dest] = 1</code>
Remove Edge	$O(1)$ — just set <code>mat[src][dest] = 0</code>
Check if edge exists	$O(1)$ — just check <code>mat[src][dest]</code>
Get all neighbors of V	$O(V)$ — scan entire row
Best for	Dense graphs, fast edge-lookup, simple code
Worst for	Sparse graphs (wastes huge memory)

3.2 Adjacency List

An Adjacency List stores each vertex with a list (or linked list) of its direct neighbors. In Python, we use a dictionary: keys are vertices, values are lists of neighbors.

Adjacency List for the same graph:

Source

1	→ [2, 4]	vertex 1 connects to: 2 and 4
2	→ [1, 4]	vertex 2 connects to: 1 and 4
3	→ [4, 5]	vertex 3 connects to: 4 and 5
4	→ [1, 2, 3, 5]	vertex 4 connects to: 1, 2, 3 and 5
5	→ [3, 4]	vertex 5 connects to: 3 and 4

In Python: { 1: [2, 4], 2: [1, 4], 3: [4, 5], 4: [1, 2, 3, 5], 5: [3, 4] }

Each list tells you: 'If I start at this vertex, I can directly reach these.'

Aspect	Adjacency List
Space Complexity	$O(V + E)$ — only stores existing edges
Add Edge	$O(1)$ — append to list
Remove Edge	$O(\text{degree})$ — scan neighbor list

Check if edge exists	$O(\text{degree})$ — scan neighbor list
Get all neighbors of V	$O(\text{degree})$ — direct from list
Best for	Sparse graphs, dynamic graphs, memory-efficient storage
Worst for	Frequent edge existence checks (slower than matrix)

□ Matrix vs List — When to Choose

Use Adjacency MATRIX when: graph is dense (E close to V^2), or you need $O(1)$ edge lookup.

Use Adjacency LIST when: graph is sparse ($E \ll V^2$), or graph grows dynamically.

Interview rule of thumb: social networks → list (millions of users, few connections each).
road maps → matrix (finite cities, many direct connections).

Feature	Adjacency Matrix	Adjacency List
Storage	$O(V^2)$	$O(V + E)$
Add Edge	$O(1)$	$O(1)$
Remove Edge	$O(1)$	$O(\text{degree})$
Check Edge Exists	$O(1)$	$O(\text{degree})$
Get All Neighbors	$O(V)$	$O(\text{degree})$
Best For	Dense graphs	Sparse graphs
Code Complexity	Simple	Slightly more complex

□ 4. Python Implementation — Adjacency Matrix

Full code with line-by-line explanation

4. Adjacency Matrix Implementation (Python)

4.1 Class Structure

We build a Graph class that internally stores the adjacency matrix as a list of lists. The constructor takes the number of vertices and creates a $V \times V$ grid of zeros.

```
class Graph:
    def __init__(self, vertex):
        # Create a V x V matrix filled with 0s
        # self.size = number of vertices
        # [[0]*vertex for x in range(vertex)]
        # → creates 'vertex' rows, each with 'vertex' zeros
        self.mat = [[0] * vertex for x in range(vertex)]
        self.size = vertex

# Example: Graph(4) creates:
# mat = [[0, 0, 0, 0],
#        [0, 0, 0, 0],
#        [0, 0, 0, 0],
#        [0, 0, 0, 0],
```

```
# [0, 0, 0, 0]
```

4.2 add_edge — Adding an Edge

Adding an edge sets `mat[src][dest] = 1`. For an undirected graph, we ALSO set `mat[dest][src] = 1` (two-way connection). A boundary check prevents out-of-bounds errors.

```
def add_edge(self, src, dest):
    # — BOUNDARY CHECK —
    # Both src and dest must be valid indices: 0 <= index < size
    # Without this, mat[src][dest] could throw IndexError
    if (0 <= src < self.size) and (0 <= dest < self.size):

        # Set edge: src → dest (forward direction)
        self.mat[src][dest] = 1

        # Set edge: dest → src (backward direction)
        # BOTH lines = UNDIRECTED graph (two-way road)
        # Remove mat[dest][src] line → becomes DIRECTED graph
        self.mat[dest][src] = 1

    else:
        print('Invalid Edge')

# For WEIGHTED graph: replace = 1 with = weight
# def add_edge(self, src, dest, weight):
#     self.mat[src][dest] = weight
#     self.mat[dest][src] = weight
```

4.3 remove_edge — Removing an Edge

Removing an edge is the reverse — set `mat[src][dest]` and `mat[dest][src]` back to 0.

```
def remove_edge(self, src, dest):
    # Same boundary check as add_edge
    if (0 <= src < self.size) and (0 <= dest < self.size):

        # Remove edge in both directions (undirected graph)
        self.mat[src][dest] = 0
        self.mat[dest][src] = 0

    else:
        print('Invalid Edge')
```

4.4 print_graph — Visualizing the Matrix

```
def print_graph(self):
    # Iterate each row and print all values separated by spaces
    for row in self.mat:
        print(' '.join(map(str, row)))

# Output for a 5-vertex graph after adding edges (0,1), (0,2), (1,3):
# 0 1 1 0 0
# 1 0 0 1 0
# 1 0 0 0 1
# 0 1 0 0 1
```

```
# 0 0 1 1 0
```

4.5 Complete Code + Step-by-Step Trace

```
# =====
# Graph - Adjacency Matrix (Complete)
# =====
class Graph:
    def __init__(self, vertex):
        self.mat = [[0] * vertex for x in range(vertex)]
        self.size = vertex

    def add_edge(self, src, dest):
        if (0 <= src < self.size) and (0 <= dest < self.size):
            self.mat[src][dest] = 1
            self.mat[dest][src] = 1
        else:
            print('Invalid Edge')

    def remove_edge(self, src, dest):
        if (0 <= src < self.size) and (0 <= dest < self.size):
            self.mat[src][dest] = 0
            self.mat[dest][src] = 0
        else:
            print('Invalid Edge')

    def print_graph(self):
        for row in self.mat:
            print(' '.join(map(str, row)))

# =====
# STEP-BY-STEP TRACE
# =====
G = Graph(5)

G.add_edge(0, 1)    # mat[0][1]=1, mat[1][0]=1
G.add_edge(0, 2)    # mat[0][2]=1, mat[2][0]=1
G.add_edge(1, 3)    # mat[1][3]=1, mat[3][1]=1
G.add_edge(2, 4)    # mat[2][4]=1, mat[4][2]=1
G.add_edge(3, 4)    # mat[3][4]=1, mat[4][3]=1
G.add_edge(2, 3)    # mat[2][3]=1, mat[3][2]=1

# Matrix after all additions:
# 0 1 1 0 0
# 1 0 0 1 0
# 1 0 0 1 1
# 0 1 1 0 1
# 0 0 1 1 0

G.remove_edge(2, 3) # mat[2][3]=0, mat[3][2]=0

# Matrix after remove_edge(2,3):
# 0 1 1 0 0
# 1 0 0 1 0
# 1 0 0 0 1  ← position [2][3] is now 0
# 0 1 0 0 1  ← position [3][2] is now 0
# 0 0 1 1 0
```

□ 5. Python Implementation — Adjacency List

Dynamic, memory-efficient graph using a dictionary

5. Adjacency List Implementation (Python)

The Adjacency List uses a Python dictionary where each key is a vertex and each value is a list of its direct neighbors. This structure grows dynamically — vertices and edges can be added on the fly.

5.1 Class Structure + `add_vertex`

```
class Graph:
    def __init__(self):
        # Dictionary: key = vertex, value = list of neighbors
        # e.g. { 1: [2, 3], 2: [1, 4], 3: [1] }
        self.adj_list = {}

    def add_vertex(self, vertex):
        # Only add if vertex does not already exist
        # (Prevents overwriting an existing vertex's edge list)
        if vertex not in self.adj_list:
            self.adj_list[vertex] = [] # start with empty neighbor list
```

5.2 `add_edge` — Both Directions

When adding an edge, we first ensure both vertices exist (calling `add_vertex` for each). Then we append each vertex to the other's neighbor list — but only if it is not already there.

```
def add_edge(self, src, dest):
    # Step 1: Ensure both vertices exist in adj_list
    # add_vertex is safe to call even if vertex exists (has guard inside)
    self.add_vertex(src)
    self.add_vertex(dest)

    # Step 2: Add edge src --> dest
    # Check first to avoid duplicate entries in the list
    if dest not in self.adj_list[src]:
        self.adj_list[src].append(dest)

    # Step 3: Add reverse edge dest --> src (undirected graph)
    # Remove this block to make it a DIRECTED graph
    if src not in self.adj_list[dest]:
        self.adj_list[dest].append(src)

# Example: add_edge(1, 2)
# adj_list = { 1: [2], 2: [1] }
# add_edge(1, 3)
# adj_list = { 1: [2, 3], 2: [1], 3: [1] }
```

5.3 `remove_edge`

```
def remove_edge(self, src, dest):
    # First verify both vertices actually exist
    if src in self.adj_list and dest in self.adj_list:

        # Remove dest from src's neighbor list
```

```

        if dest in self.adj_list[src]:
            self.adj_list[src].remove(dest)

        # Remove src from dest's neighbor list (undirected)
        if src in self.adj_list[dest]:
            self.adj_list[dest].remove(src)
    else:
        print('One or both vertices not found')

```

5.4 remove_vertex

Removing a vertex requires two steps: remove all references to it from other vertices' neighbor lists, then delete the vertex's own entry from the dictionary.

```

def remove_vertex(self, vertex):
    if vertex not in self.adj_list:
        print('Vertex does not exist')
        return

    # Step 1: Remove this vertex from every other vertex's list
    # We must do this BEFORE deleting the vertex itself
    for v in self.adj_list:
        if vertex in self.adj_list[v]:
            self.adj_list[v].remove(vertex)

    # Step 2: Delete the vertex's own entry from dictionary
    del self.adj_list[vertex]

# Example: remove_vertex(2) from { 1:[2,3], 2:[1,4], 3:[1], 4:[2] }
# Step 1: scan all lists, remove '2' from [1:[2,3] → 1:[3]], [4:[2] → 4:[]]
# Step 2: del adj_list[2]
# Result: { 1: [3], 3: [1], 4: [] }

```

5.5 print_graph

```

def print_graph(self):
    for vertex in self.adj_list:
        print(f'{vertex} --> {self.adj_list[vertex]}')

# Output:
# 1 --> [2, 3]
# 2 --> [1, 4]
# 3 --> [1, 5]
# 4 --> [2, 5]
# 5 --> [3, 4]

```

5.6 Complete Code + Testing

```

# =====
# Graph - Adjacency List (Complete)
# =====
class Graph:
    def __init__(self):
        self.adj_list = {}
    def add_vertex(self, vertex):
        if vertex not in self.adj_list:
            self.adj_list[vertex] = []
    def add_edge(self, src, dest):
        self.add_vertex(src);
        self.add_vertex(dest)

```

```

        if dest not in self.adj_list[src]: self.adj_list[src].append(dest)
        if src not in self.adj_list[dest]: self.adj_list[dest].append(src)
    def remove_edge(self, src, dest):
        if src in self.adj_list and dest in self.adj_list:
            if dest in self.adj_list[src]: self.adj_list[src].remove(dest)
            if src in self.adj_list[dest]: self.adj_list[dest].remove(src)
    def remove_vertex(self, vertex):
        if vertex not in self.adj_list: return
        for v in self.adj_list:
            if vertex in self.adj_list[v]: self.adj_list[v].remove(vertex)
        del self.adj_list[vertex]
    def print_graph(self):
        for v in self.adj_list: print(f'{v} --> {self.adj_list[v]}')

# ----- Testing -----
G = Graph()
G.add_edge(1, 2); G.add_edge(1, 3); G.add_edge(2, 4)
G.add_edge(3, 5); G.add_edge(4, 5)

G.print_graph()
# 1 --> [2, 3]
# 2 --> [1, 4]
# 3 --> [1, 5]
# 4 --> [2, 5]
# 5 --> [3, 4]

G.remove_edge(1, 3)
# 1 --> [2]          ← 3 removed from 1's list
# 3 --> [5]          ← 1 removed from 3's list

G.remove_vertex(2)
# 1 --> []           ← 2 removed from 1's list
# 4 --> [5]          ← 2 removed from 4's list
# vertex 2 deleted entirely

```

⚡ 6. Complete Complexity Summary

Time & Space for all operations — both representations

6. Complete Complexity Summary

Operation	Adjacency Matrix	Adjacency List	Notes
Space	$O(V^2)$	$O(V + E)$	List wins for sparse graphs
Add Vertex	$O(V^2)$ — resize matrix	$O(1)$	List is much better dynamically
Add Edge	$O(1)$	$O(1)$	Both are constant time
Remove Edge	$O(1)$	$O(\text{degree})$	Matrix wins here
Check Edge	$O(1)$	$O(\text{degree})$	Matrix wins here
Get Neighbors	$O(V)$ — scan row	$O(\text{degree})$	List wins for sparse
BFS / DFS	$O(V^2)$	$O(V + E)$	List is much faster for sparse

□ Interview Cheat Sheet — Graph Quick Reference

Graph = Vertices (nodes) + Edges (connections)
Undirected: $\text{mat}[i][j] = \text{mat}[j][i] = 1$ (symmetric)
Directed: $\text{mat}[i][j] = 1$ only (one direction)
Weighted: $\text{mat}[i][j] = \text{weight value}$
Dense graph → Adjacency Matrix | Sparse graph → Adjacency List
 V = number of vertices | E = number of edges
Degree of a vertex = number of edges connected to it

□ 7. Key Terminology Glossary

Every graph term you need to know

7. Key Terminology Glossary

Term	Definition	Example
Vertex (Node)	A single point/entity in the graph	A city, a person, a webpage
Edge	A connection between two vertices	A road, a friendship, a hyperlink
Degree	Number of edges connected to a vertex	Vertex 1 connected to 3 others → degree 3
In-degree	Number of incoming edges (directed)	3 people follow you → in-degree 3
Out-degree	Number of outgoing edges (directed)	You follow 50 people → out-degree 50
Path	Sequence of vertices connected by edges	$1 \rightarrow 2 \rightarrow 4 \rightarrow 5$
Cycle	Path that starts and ends at same vertex	$1 \rightarrow 2 \rightarrow 3 \rightarrow 1$
Connected Graph	Every vertex reachable from every other	One-piece graph, no islands
Disconnected Graph	Some vertices have no path to others	Graph with isolated components
Adjacent Vertices	Vertices directly connected by an edge	If edge (1,2) exists, 1 and 2 are adjacent
Sparse Graph	Few edges compared to V^2	Social network (you know few of all users)
Dense Graph	Many edges, close to V^2 edges	Road network in a city
Weighted Graph	Edges carry numeric values	GPS map with distances
Self-loop	Edge from a vertex to itself	Edge (3,3) → loop at vertex 3